HireHelper — Real-Time Chat Feature

Design, task division, and rollout plan for adding request-based chat between task owners and accepted helpers.

1. Feature Design: Real-Time Conversation

Chat is added as a real-time conversation between the task owner and the helper, available only after a request is accepted. A sensible implementation flow:

```
Accepted Request
      ↓
Create/find a Conversation for task + owner + helper
      ↓
Dashboard shows "Messages" / chat shortcut
      ↓
Users exchange Message records in real time
```

Backend

- Add a **Conversation** model: `task`, `participants`, `lastMessage`, timestamps.
- Add a **Message** model: `conversation`, `sender`, `text`, `readBy`, timestamps.
- When a task request is accepted, create the conversation automatically.
- Add protected APIs:
  - `GET /api/conversations`
  - `GET /api/conversations/:id/messages`
  - `POST /api/conversations/:id/messages`
  - `PATCH /api/conversations/:id/read`
- Check that the logged-in user belongs to the conversation before returning or sending anything.
- Use Socket.IO for instant message delivery and unread badges. REST APIs still load chat history.

Frontend

- Add a `features/chat/` folder with:
  - `pages/Chat.jsx`
  - `components/ConversationList.jsx`
  - `components/MessageThread.jsx`
  - `components/MessageComposer.jsx`
  - `services/chat.service.js`
- Put a Messages icon/card on the dashboard, linking to `/messages`.
- Show each conversation with task title, other participant, last message, and unread count.
- Connect Socket.IO once after login; update the open thread and notification badge when messages arrive.

> ***Key rule:*** *do not let arbitrary users chat. Only allow the task creator and the accepted helper to message for that specific task.*

> ***Recommended build order:*** *build chat using REST APIs first, then add Socket.IO after the screens and permissions are stable. This makes it easier to test and avoids making the dashboard depend on a live socket connection.*

2. Team Work Division

Divide the work by clear boundaries so people do not edit the same files.

Person	Ownership	Main work
1	**Chat backend models**	Create Conversation and Message Mongoose models; indexes and participant rules
2	**Chat backend APIs**	Build conversation list, message history, send-message, and mark-read routes/controllers
3	**Request integration**	Update “accept request” logic to create/find a conversation for the task owner and accepted helper
4	**Real-time layer**	Configure Socket.IO in Express; implement user rooms, send/receive events, online presence optional
5	**Chat interface**	Build /messages page: conversation list, message thread, composer, empty/loading states
6	**Dashboard integration**	Add Messages card/icon, unread count/badge, navigation, and notification interaction
7	**Testing and integration**	Test permissions, API response shapes, cross-user messaging, error handling; resolve merged integration issues

Branches

```
feature/chat-models
feature/chat-api
feature/chat-request-integration
feature/chat-realtime
feature/chat-ui
feature/chat-dashboard
feature/chat-testing
```

Shared Contracts (agree before coding)

```
Conversation = {
  task,
  participants: [userId, userId],
  lastMessage,
  lastMessageAt
}

Message = {
  conversation,
  sender,
  text,
  readBy: [userId]
}
```

Agreed API responses:

```
GET  /api/conversations
GET  /api/conversations/:conversationId/messages
POST /api/conversations/:conversationId/messages
PATCH /api/conversations/:conversationId/read
```

Recommended Order

1. Person 1 completes models.
2. Person 2 builds the REST API using the agreed model shapes.
3. Person 3 connects accepted requests to conversation creation.
4. Persons 5 and 6 can build UI using sample data while APIs are underway.
5. Person 4 adds Socket.IO after REST messaging works.
6. Person 7 tests the merged feature using two test accounts.

> ***Most important rule:*** *Person 3, 4, and 6 may all need shared server/dashboard files, so agree in advance who edits which exact file. Everyone else should stay inside their own feature folder to avoid merge conflicts.*

3. Building Chat Before the UI Is Finalized

You can build it without a finalized UI first. Treat chat as a working feature in two stages:

1. Build the backend and connect it to accepted requests.
2. Create a simple temporary chat screen, then restyle it once your Figma UI is ready.

Temporary /messages Layout

```
+------------------+-----------------------------------+
| Conversations    |  Task: Help Moving Furniture      |
|                  |                                   |
| Alex - 2m ago    |  Alex: Hi, I can help at 2 PM.    |
| Painting - 1h    |                                   |
|                  |            You: Great, thank you! |
|                  |                                   |
|                  | [ Type a message... ]    [ Send ] |
+------------------+-----------------------------------+
```

It only needs:

- Left panel: conversations/task names
- Right panel: messages
- Input and Send button
- A Messages link in the sidebar or dashboard

Team Division (Even Without Final UI)

- 3 people: backend models, APIs, and request-to-conversation creation
- 2 people: build the temporary React page and connect it to APIs
- 1 person: Socket.IO / real-time messages
- 1 person: testing, permissions, and later Figma styling

Use your existing app look — sidebar, cards, buttons, fonts, and colors — so it feels consistent. When the design arrives, the UI team changes only JSX/CSS; the backend and chat logic remain unchanged.